DeepLabv3

Building Blocks for Robust Segmentation Models

7 min read · May 30, 2023

In this post we explore the main concepts of DeepLabV3 and aim to understand it's fundamental building blocks. We will see what each of these blocks do and how they can be combined together to create the architecture. Understanding these basic blocks will carryover into understanding more complex segmentation architectures and provide intuition for training your own custom architectures.

Here's an overview of what we will learn

- **Introduction** — *What is DeepLabv3?*
- **Main Ideas** — *What are the essential building blocks?*
- **Architecture** — *How do we combine these building blocks?*
- **Implementation** — *If you're here for code, this one's for you*
- **Conclusion** — *tl;dr*

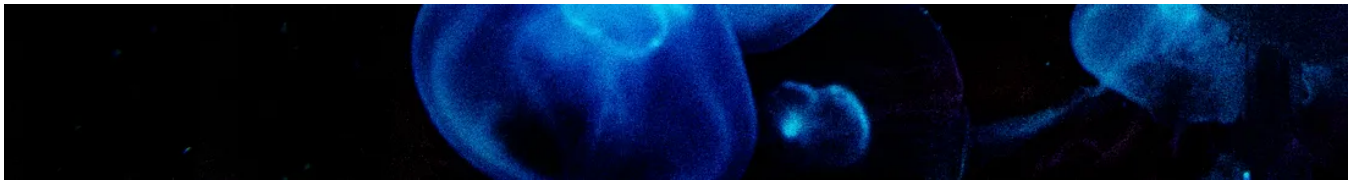


Photo by Nicole Avagliano on Unsplash

Introduction

DeepLabv3 is a Deep Neural Network (DNN) architecture for Semantic Segmentation Tasks. It uses Atrous (Dilated) Convolutions to control the receptive field and feature map resolutions without increasing the total number of parameters. Another main attribute is something called Atrous Spatial Pyramid Pooling which effectively extracts multi-scale features that contain useful information for segmentation. In general, the network is able to capture dense feature maps with rich long-range information that can be used to accurately segment images.

Main Ideas

Deep and Fully Convolutional Neural Networks have been shown to be effective for segmentation tasks. Typically an encoder is used to encode the input image into a compressed representation and a decoder is used to upsample these features to the desired resolution. There are typically skip connections between the Encoder and Decoder to pass expressive high level information throughout the network. See figure 1 for an example.

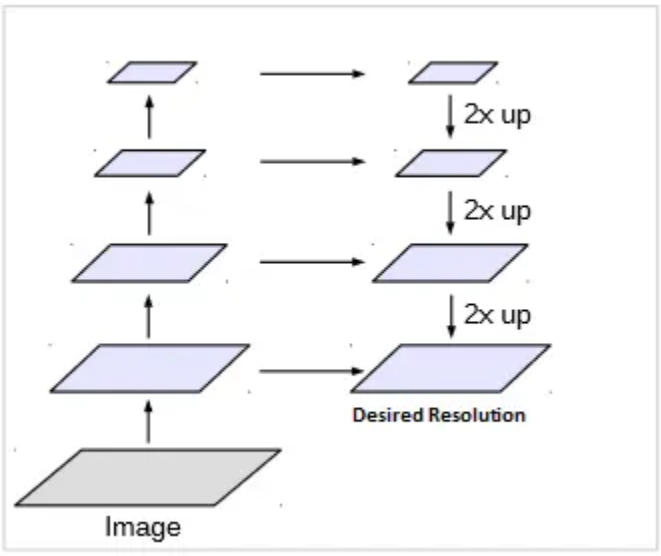


Figure 1. Encoder-Decoder Architecture. Modified from Source.

The encoder typically uses repeated max-pooling and striding operations to obtain a compressed representation at a significantly reduced resolution. The DeepLab architecture proposes a different approach where atrous convolution blocks are used to obtain finer resolution feature maps and bilinear upsamling is used to obtain the desired resolution.

Atrous Convolution

Atrous Convolution (same as Dilated Convolution) is the cornerstone of the DeepLab architecture. In atrous convolution, we just insert zeros into the convolution kernel to increase the size of the kernel without increasing the number of learnable parameters (because we don't care about the zeros).

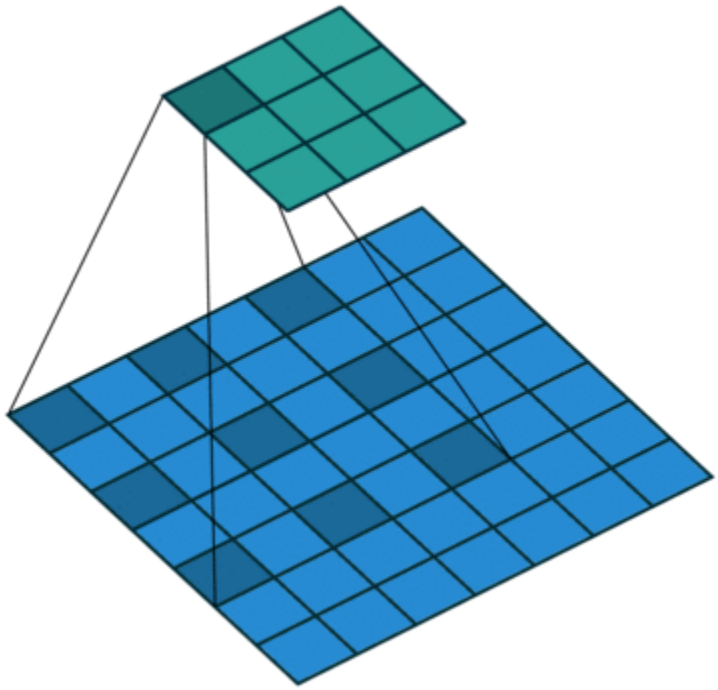


Figure 2. Atrous Convolution. Source.

In figure 2, we can see that the 3x3 atrous kernel has a 5x5 receptive field. If we were to stack up layers of atrous convolution, we would not only have a large receptive field, we would have more dense feature maps than with regular convolutions. See figure 3.

$$y[i] = \sum_{k=1}^{K} x[i + r \cdot k]w[k]$$

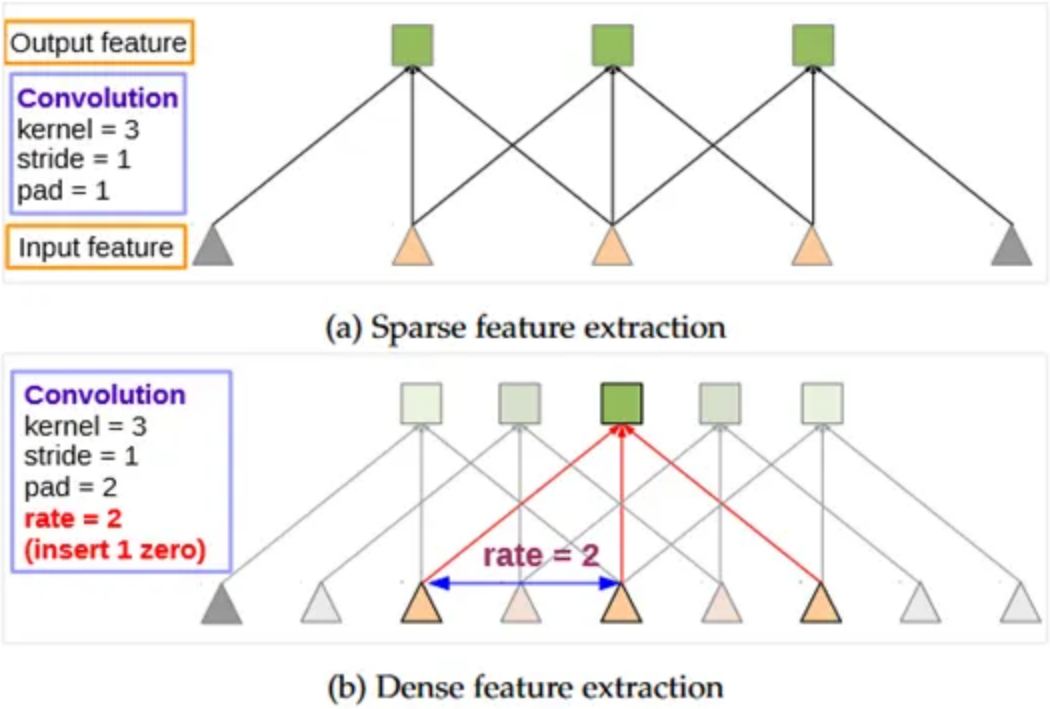


Figure 3. 1D Atrous Convolution for dense feature extraction. Source.

At the top of figure 3, we can recognize that Atrous convolution is a generalization of regular convolution, where the rate r determines the number of zeros to insert. In regular convolution $r = 1$.

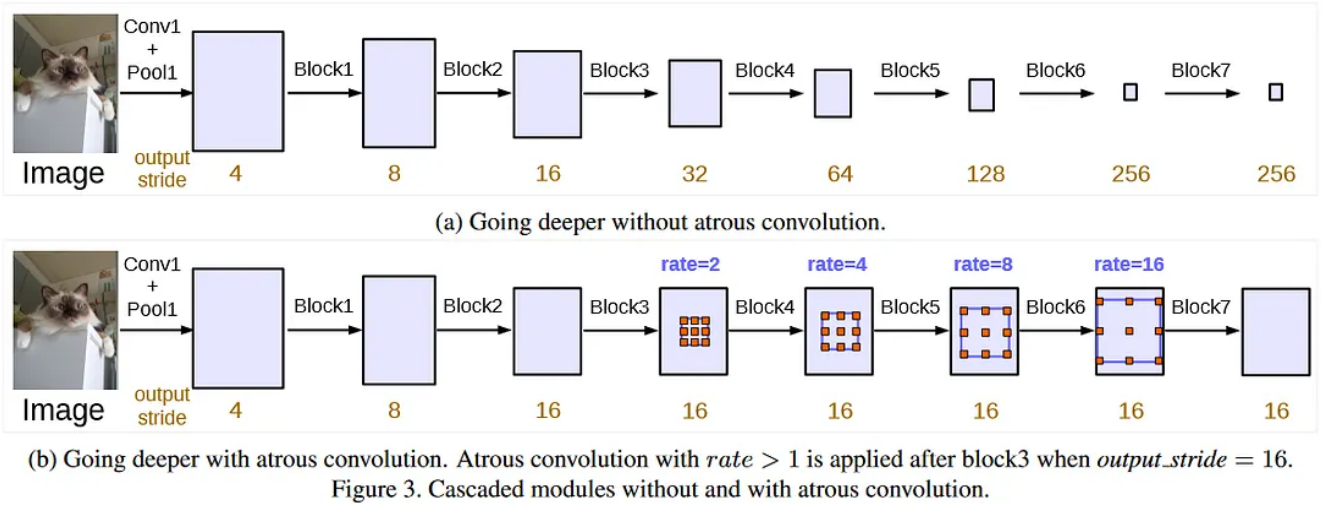
Atrous Convolution has the benefits of:

- Enabling more dense features to be extracted at deeper levels
- Allows for control over receptive field via rate
- Retains same number of learnable parameters as regular convolution

How exactly does Atrous Convolution help us with segmentation?

Atrous Convolution helps us construct a deeper network that retains more high level information at finer resolutions without increasing the number of parameters. See figure 4, where the *output stride* is defined as the ratio between the input and output

image. A network with a higher output stride will be able to extract better and higher resolution features.



(a) Going deeper without atrous convolution.

(b) Going deeper with atrous convolution. Atrous convolution with $rate > 1$ is applied after block3 when $output\_stride = 16$.

Figure 3. Cascaded modules without and with atrous convolution.

Figure 4. Encoders with (top) and without (bottom) atrous convolution. Source.

Notice that in the Atrous architecture a decoder does not need to upsample from extremely decimated feature maps. By using atrous convolution, we are constructing a backbone that can extract fine resolution feature maps.

> ***A drawback of atrous convolutions:*** *Atrous convolutions enable large features maps to be extracted deep in the network at the cost of increased memory consumption. An evident symptom will be a quick overload of the GPU capacity. Additionally, the inference times will be longer. The tradeoff is the we obtain a powerful model in lieu of speed.*

Multi-Grid Method

DeepLab employs something called the multi-grid method, where different atrous convolution rates are applied to different blocks of the network. See bottom of figure 4, where the rates are increased as information flows deeper into the network.

Atrous Spatial Pyramid Pooling

If Atrous Convolution is the cornerstone then Atrous Spatial Pyramid Pooling (ASPP) is the foundation.

Spatial Pyramid Pooling (SPP) resamples features at multiple scales and then pools them together (usually with an Average Pooling layer).

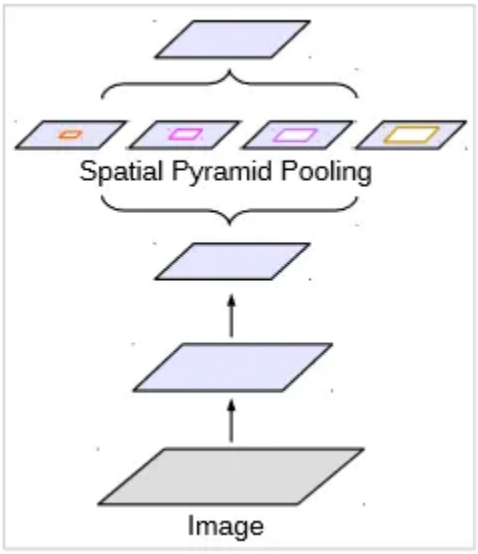


Figure 5. Spatial Pyramid Pooling. Source.

In the case of ASPP, the feature scales are changed via Atrous Convolution rate. One thing to note with this, is that when the rate gets too large, the Atrous Convolution essentially becomes a 1x1 convolution. In this case, the rate is close to the size of the feature map and context from the entire image can not be captured. To overcome this problem, a 1x1 convolution is applied which retains the original feature map shape, and consequently obtains information from the entire feature map. The outputs are concatenated and Global Average Pooling is then applied.

Architecture

Now we will see how all the pieces fit together to form the underlying blocks of the DeepLabv3 Architecture. Figure 6 shows the basic architecture of a DeepLabv3 network, where the main blocks are just the backbone and the head. Each of the

main blocks is comprised of sub-blocks.

> *NOTE: While backbone and head are common terminology for neural network architectures, the sub-block block names are not necessarily universal. The important part is to understand the underlying concepts so that you can apply them to any deep architecture.*

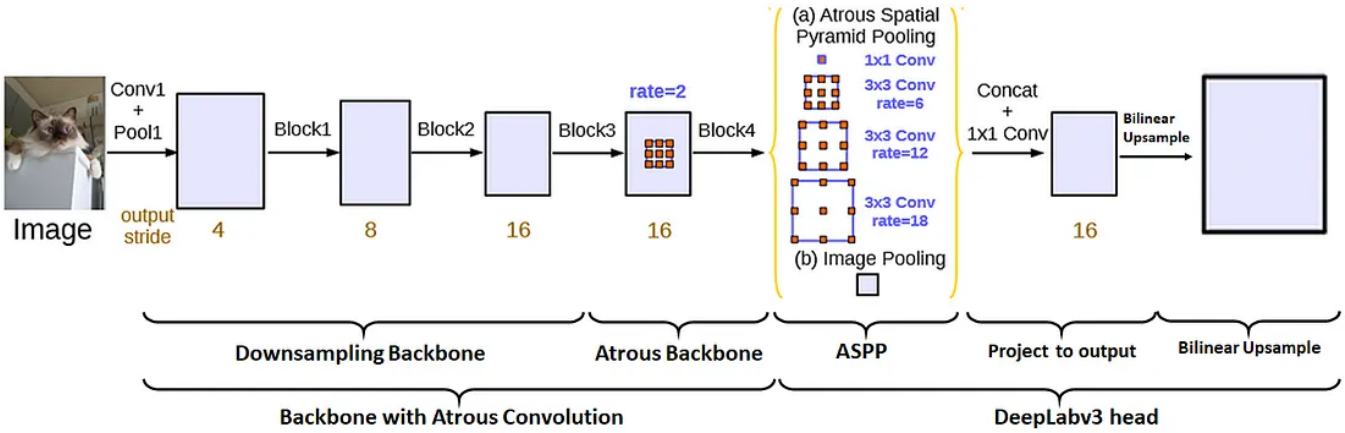


Figure 6. DeepLabv3 Architecture with labeled blocks. Modified from Source.

The overall backbone encodes image features into rich high resolution feature maps. The down sampling backbone takes the input image and extracts *shallow features*, while the Atrous backbone encodes *deep features* at a high resolution without increasing the total number of parameters.

In the second part of the network, the DeepLabv3 head is applied to the end of the backbone to produce the outputs. This head first comprises of a ASPP block that resamples features at different scales and pools them together providing high quality multi-scale information. After the ASPP block we have an additional block that essentially projects the feature maps to the desired number of segmentation classes. Lastly, bilinear upsampling is used to obtain feature maps at the same resolution as the input image.

Implementation

Here, we will briefly cover practical implementations for DeepLabv3.

The backbone (sometimes called an encoder) is usually a modified version of an ImageNet model such as a ResNet or MobileNet, but we can could really use any

type of network as long as we apply atrous convolution to the final layers to get fine resolution feature maps. Even though we change the architecture by dilating some of the convolutions, we don't change any of the weights so we can still use the pre-trained weights with no issue. It is also important to prepare the input in the same way that the backbone was trained.

We could write code for the DeepLabv3 head ourselves (it would be a good exercise!), but thankfully torchvision has both pre-trained backbones and pretrained heads, here's the link to the docs. Let's see an example.

```
from torchvision.models.segmentation import deeplabv3_resnet50

deeplabv3 = deeplabv3_resnet50(
    weights='COCO_WITH_VOC_LABELS_V1',
    weights_backbone='IMAGENET1K_V1'
)

# change outputs to desired number of classes
deeplabv3.classifier[4] = torch.nn.Conv2d(256, num_classes, kernel_size=(1, 1),
```

We can also use Segmentation Models Pytorch, which supports a wide variety of pre-trained backbones/encoders, but the segmentation heads don't seem to be pre-trained.

```
import segmentation_models_pytorch as smp

deeplabv3 = smp.DeepLabV3(
    encoder_name='timm-mobilenetv3_small_100',
    encoder_weights='imagenet',
    classes=num_classes
)
```

Conclusion

The DeepLabv3 Architecture is composed of two main blocks: a **backbone** that is

able to provide fine resolution feature maps via Atrous Convolution and a **DeepLabv3 Head** that is able to extract multi-scale features at a fine resolution, projects them to the desired number of feature maps (number of segmentation classes), and upsamples them to the input image resolution.

Since DeepLabv3 has a modular architecture, we can mix and match different blocks to get the desired performance. For example we can use a pre-trained ResNet101 backbone for high performance, or we can forgo some accuracy for speed and use a MobileNet backbone instead. We can even add multiple heads to perform multi-task learning, *e.g. perform segmentation and depth estimation at the same time.*

References

[1] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, 'Rethinking Atrous Convolution for Semantic Image Segmentation', *CoRR*, vol. abs/1706.05587, 2017. Retrieved from http://arxiv.org/abs/1706.05587

[2] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, 'DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs', *CoRR*, vol. abs/1606.00915, 2016. Retrieved from https://arxiv.org/pdf/1606.00915.pdf

Thanks for Reading! If you found this useful please consider clapping 👏 and following for more.

Ready to join the cutting edge?

Deep Learning | Artificial Intelligence | Data Science | Segmentation | Computer Vision

Follow

Written by Isaac Berrios

754 followers · 205 following

Electrical Engineer interested in Sensors and Autonomy. I write about Sensors, Signal Processing, Computer Vision, and Embedded Computing.